المدرسة الوطنية للعلوم التطبيقية
ⵜⵉⵎⵙⵙⵓⵜ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵙⵙⴰⵏⵉⵡⵉⵏ ⵜⵉⵜⵜⴰⵡⵉⵏ
Ecole Nationale des Sciences Appliquées de Fès

جامعة سيدي محمد بن عبد الله
ⵜⴰⵙⴷⴰⵡⵉⵜ ⵙⵉⴷⵉ ⵎⵓⵃⴰⵎⵎⴷ ⴱⵏ ⵄⴱⴷⵍⵍⴰⵀ
Université Sidi Mohamed Ben Abdellah

TP 4 :

AWS Academy Cloud Foundations





BELHAJ KHAOULA

N 9

GSCSI- G1

Table des matières

Table des matières 2
Table de figures 4
Atelier 6 : Mise à l'échelle et équilibrage des charges de votre architecture 5
Introduction 5
Tâche 1 : création d'une AMI pour Auto Scaling 6
1. Introduction à la tâche 6
2. Procédure technique 6
3. Rôle de l'AMI dans l'architecture finale 6
4. Conclusion de la tâche 7
Tâche 2 : création d'un équilibreur de charge 8
1. Introduction à la tâche 8
2. Déploiement technique : Les trois piliers 8
3. Analyse de l'état final 10
4. Conclusion de la tâche 10
Tâche 3 : création d'un modèle de lancement et d'un groupe Auto Scaling 11
1. Introduction à la tâche 11
2. Configuration du Modèle de lancement (Launch Template) : `LabConfig` 11
3. Configuration du Groupe Auto Scaling : `Lab Auto Scaling Group` 12
4. Intégration et Notification 13
5. Conclusion de la tâche 14
Tâche 4 : vérification du fonctionnement de l'équilibrage de charge 15
1. Introduction à la tâche 15
2. Validation des instances et de leur santé 15
3. Test de connectivité via le nom DNS 16
4. Conclusion de la tâche 17
Tâche 5 : test d'Auto Scaling 18
1. Introduction à la tâche 18
2. Déclenchement de l'élasticité : Le Test de Charge 18
3. Analyse des résultats 20
4. Conclusion de la tâche 20
Tâche 6 : résiliation de l'instance Web Server 1 21

1. Introduction à la tâche 21
2. Procédure de nettoyage 21
3. Analyse du résultat : La "Haute Disponibilité" en action 21
4. Conclusion de la tâche et de l'atelier 22
Conclusion 23

Table de figures

Figure 1 création d'une Amazon Machine Image 7
Figure 2 LabGroup 8
Figure 3 LabELB 9
Figure 4 Trafic HTTP 10
Figure 5 LabConfig 12
Figure 6 Utilisation CPU à 60 % 13
Figure 7 Identification 13
Figure 8 Lab Auto Scaling Group 14
Figure 9 Apparition de deux nouvelles instances 15
Figure 10 Health Checks 16
Figure 11 Application web 16
Figure 12 CloudWatch: Alarmes 18
Figure 13 Load Test 19
Figure 14 Lab Instance 20
Figure 15 Résilier l'instance 22
Figure 16 Résultat 23

Atelier 6 : Mise à l'échelle et équilibrage des charges de votre architecture

Introduction

Dans le paysage actuel de l'informatique en nuage (Cloud Computing), la haute disponibilité et la résilience sont devenues des impératifs critiques pour toute application d'entreprise. Cet atelier se concentre sur l'implémentation d'une architecture capable de s'adapter dynamiquement aux variations de charge de travail tout en garantissant une expérience utilisateur fluide.

Le cœur de cet exercice repose sur la synergie entre deux services piliers d'Amazon Web Services : **Elastic Load Balancing (ELB)** et **Auto Scaling**. L'objectif est de transformer une infrastructure statique, reposant sur un serveur unique, en une infrastructure élastique et tolérante aux pannes. Nous explorons ici comment automatiser la répartition du trafic réseau et comment piloter le cycle de vie des instances Amazon EC2 en fonction de métriques de performance réelles (comme l'utilisation du processeur). Cette approche permet non seulement d'optimiser les performances lors des pics d'activité, mais également de rationaliser les coûts opérationnels en réduisant les ressources inutilisées pendant les périodes calmes.

Tâche 1 : création d'une AMI pour Auto Scaling

1. Introduction à la tâche

La création d'une **Amazon Machine Image (AMI)** est une étape critique dans le cycle de vie du déploiement cloud. Dans cet atelier, l'objectif était de capturer l'état exact du serveur **Web Server 1** (incluant le système d'exploitation, les configurations logicielles et le code de l'application) pour en faire un "modèle" réutilisable.

Sans cette AMI, le service Auto Scaling ne saurait pas quoi installer sur les nouvelles instances qu'il lance pour répondre à la charge. L'AMI garantit une **homogénéité parfaite** du parc informatique : chaque serveur ajouté sera le clone conforme du premier.

2. Procédure technique

L'opération a suivi un protocole précis pour garantir l'intégrité des données :

- **Vérification de l'état (Health Check) :** Avant la capture, nous avons attendu que l'instance affiche le statut `2/2 checks passed`. Cela assure que le système de fichiers est stable et que les services réseau sont opérationnels.
- **Capture de l'image :** Via le menu `Actions > Image et modèles`, nous avons déclenché la fonction `Créer une image`.
- **Paramétrage :**
    - **Nom :** `WebServerAMI` (Identifiant unique pour le modèle de lancement).
    - **Description :** `Lab AMI for Web Server` (Documentation de l'usage de l'image).
- **Mécanisme sous-jacent :** AWS crée instantanément un **instantané (Snapshot)** du volume Amazon EBS attaché à l'instance pour sauvegarder toutes les données du disque dur.

3. Rôle de l'AMI dans l'architecture finale

L'AMI `WebServerAMI` créée lors de cette tâche joue trois rôles stratégiques :

1. **Rapidité de mise à l'échelle :** Contrairement à une installation manuelle qui prendrait des dizaines de minutes, le lancement d'une instance à partir d'une AMI se fait en moins d'une minute.
2. **Immuabilité :** Le serveur original (`Web Server 1`) peut être supprimé sans crainte , car son "essence" est sauvegardée dans l'AMI.
3. **Standardisation :** Toutes les instances du groupe Auto Scaling (les `Lab Instances`) partagent la même configuration de sécurité et les mêmes fichiers sources, éliminant les erreurs de configuration individuelles.

4. Conclusion de la tâche

La Tâche 1 s'est achevée avec succès lors de l'obtention de l'**ID d'AMI (ami-07647154e440f94b9)** . Cette ressource est désormais stockée dans votre catalogue privé AWS. Elle a permis de passer d'une infrastructure dépendante d'un seul serveur physique virtuel à une architecture basée sur un **modèle logiciel**, ouvrant la voie à l'automatisation complète réalisée dans les tâches suivantes (Modèle de lancement et Auto Scaling).

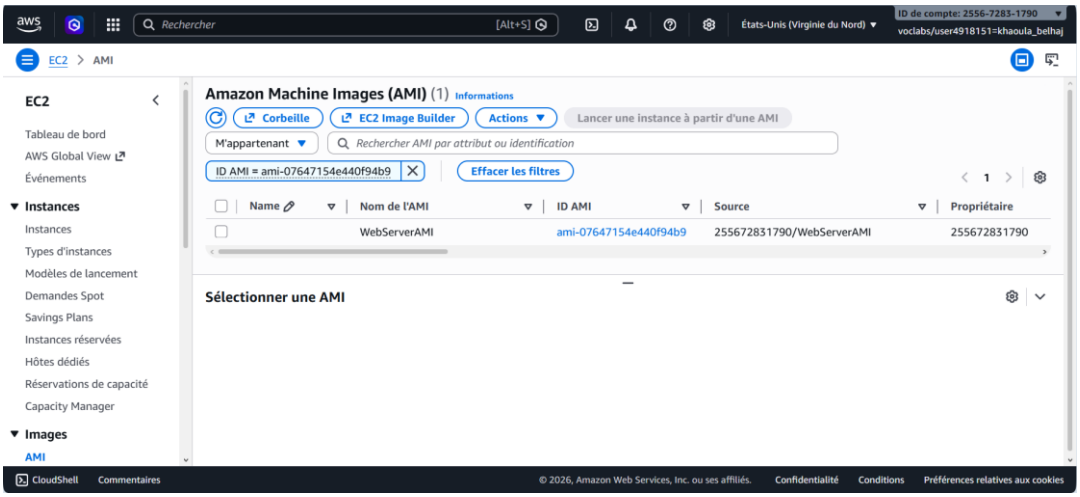


Figure 1 création d'une Amazon Machine Image

Tâche 2 : création d'un équilibreur de charge

1. Introduction à la tâche

L'objectif de cette deuxième étape est de passer d'une connexion directe (point à point) à une architecture distribuée. L'**Application Load Balancer (ALB)** agit comme un chef d'orchestre : il reçoit toutes les requêtes HTTP entrantes et les distribue intelligemment vers les serveurs disponibles.

Cette configuration est essentielle pour garantir que si un serveur tombe en panne ou devient surchargé, l'utilisateur final ne subisse aucune interruption de service. C'est le point d'entrée unique (Single Point of Contact) de votre application.

2. Déploiement technique : Les trois piliers

Le succès de cette tâche a reposé sur la configuration de trois composants interdépendants :

A. Le Groupe Cible (Target Group) : `LabGroup`

Avant de créer l'équilibreur, nous avons défini sa destination.

- **Fonction :** Regrouper les instances logiques qui traiteront les requêtes.
- **Vérification de l'état (Health Checks) :** Le groupe cible a été configuré pour surveiller la santé des instances. Si une instance ne répond pas, l'équilibreur cesse immédiatement de lui envoyer du trafic.

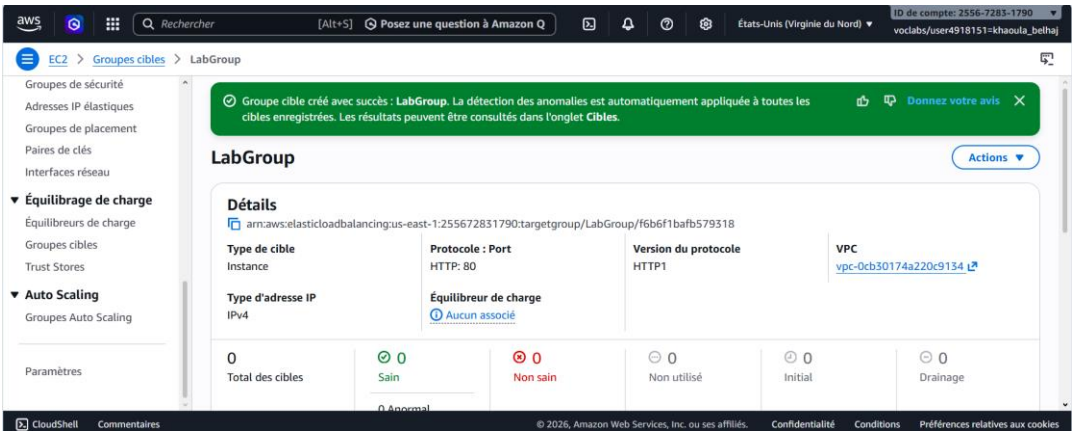


Figure 2 LabGroup

B. Le réseau et la Haute Disponibilité

L'équilibreur `LabELB` a été déployé dans le **Lab VPC** avec une configuration multi-zones :

- **Sous-réseaux publics :** Nous avons sélectionné `Public Subnet 1` et `Public Subnet 2`.
- **Justification :** L'ELB doit être dans des sous-réseaux publics pour être accessible depuis Internet, mais il doit couvrir deux zones de disponibilité différentes pour assurer la redondance en cas de panne d'un centre de données complet d'AWS.

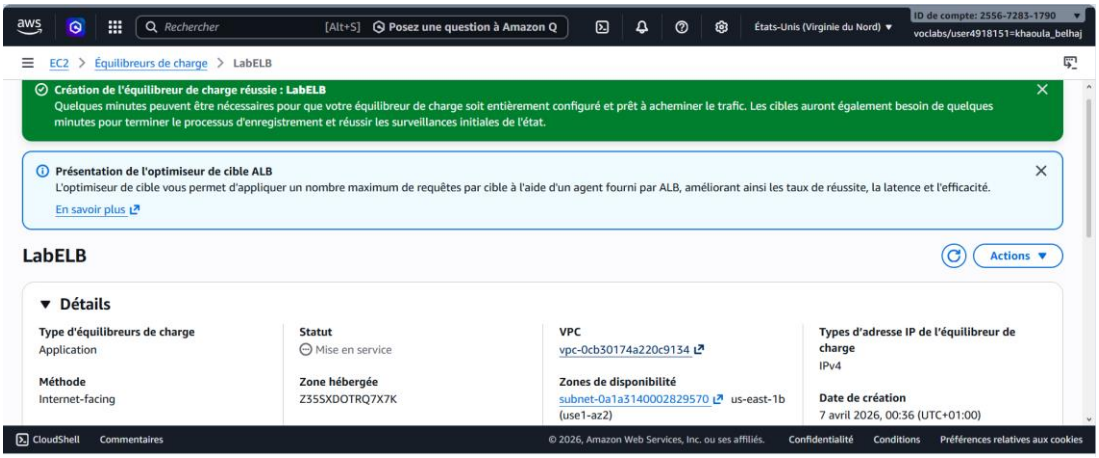


Figure 3 LabELB

C. La Sécurité et l'Écoute (Listeners)

- **Groupe de sécurité :** Nous avons appliqué le `Groupe de sécurité web`, autorisant uniquement le trafic HTTP standard.
- **L'écouteur (Listener) :** Un règle a été créée sur le port **80**. Elle stipule que tout trafic arrivant sur ce port doit être "transféré" (Forward) vers notre `LabGroup`.

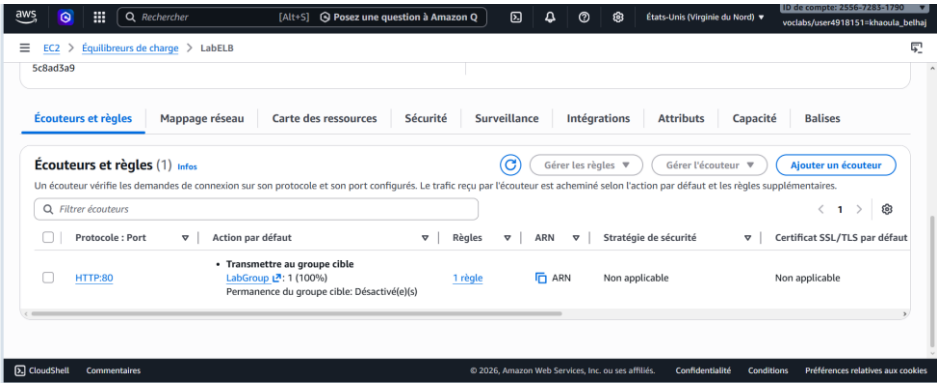


Figure 4 Trafic HTTP

3. Analyse de l'état final

À la fin de cette tâche, l'équilibreur de charge est passé à l'état `Provisioning` (Mise en service).

- **Nom DNS :** L'ALB génère une adresse URL unique (ex: `LabELB-12345.us-west-2.elb.amazonaws.com`). C'est cette adresse, et non plus l'IP d'un serveur spécifique, qui devient l'URL officielle de l'application.
- **Abstraction :** L'infrastructure est désormais prête à accueillir un nombre variable d'instances en arrière-plan sans que l'utilisateur n'ait à changer d'adresse de connexion.

4. Conclusion de la tâche

La Tâche 2 a permis de découpler l'accès utilisateur de l'infrastructure de calcul. En mettant en place le **LabELB**, nous avons posé la brique logicielle nécessaire à la **tolérance aux pannes**. Même si nous n'avions pas encore d'instances actives dans le groupe cible à ce stade, la "porte d'entrée" sécurisée et intelligente était configurée pour diriger le trafic vers les futures ressources qui seront créées par Auto Scaling dans la tâche suivante.

Tâche 3 : création d'un modèle de lancement et d'un groupe Auto Scaling

1. Introduction à la tâche

Cette phase constitue le "cerveau" de votre architecture élastique. L'objectif était de passer d'une gestion manuelle des serveurs à une gestion **déclarative**.

Au lieu de lancer chaque instance une par une, nous avons défini un **Modèle de lancement** (ce qu'il faut lancer) et un **Groupe Auto Scaling** (combien d'unités lancer et quand). Cette automatisation garantit que l'infrastructure s'ajuste d'elle-même pour maintenir les performances tout en optimisant les coûts.

2. Configuration du Modèle de lancement (Launch Template) :

`LabConfig`

Le modèle de lancement sert de "fiche technique" pour chaque nouvelle instance créée par AWS :

- **Identité :** Utilisation de la `WebServerAMI` créée en Tâche 1.
- **Ressources :** Type d'instance `t2.micro` (équilibre entre coût et performance pour le lab).
- **Sécurité :** Association de la paire de clés `vockey` et du `Groupe de sécurité web`.
- **Surveillance accrue :** Activation de la **surveillance CloudWatch détaillée**. Cette option est cruciale car elle permet d'obtenir des métriques toutes les minutes (au lieu de 5), offrant une réactivité quasi immédiate aux pics de charge.

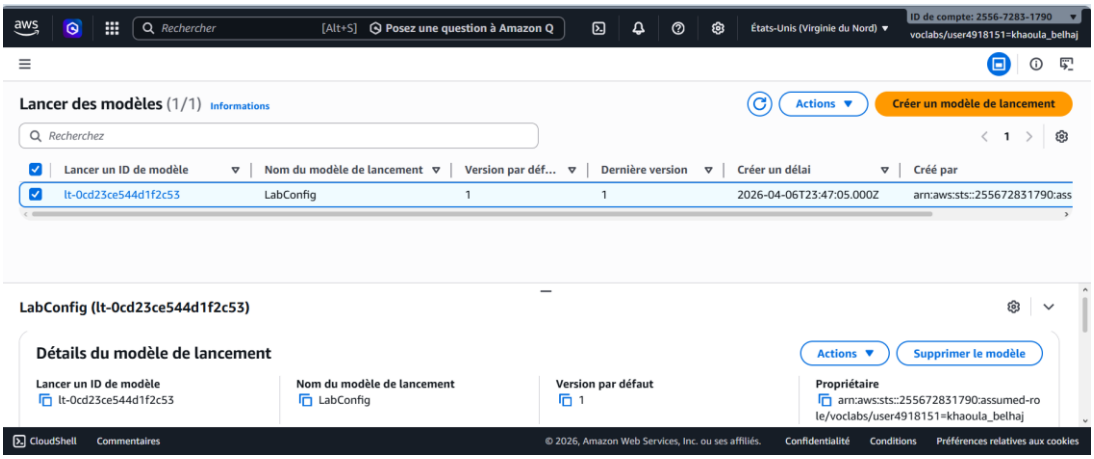


Figure 5 LabConfig

3. Configuration du Groupe Auto Scaling : `Lab Auto Scaling Group`

C'est ici que les règles de gestion du parc de serveurs sont édictées :

A. Réseau et Disponibilité

Le groupe a été configuré pour déployer des instances dans le **VPC de l'atelier** au sein de deux sous-réseaux privés (`Private Subnet 1` et `Private Subnet 2`).

Note stratégique : Placer les instances dans des sous-réseaux privés renforce la sécurité, car elles ne sont pas directement accessibles depuis Internet. Seul l'équilibreur de charge (créé en Tâche 2) peut leur parler.

B. Dimensionnement et Politiques de mise à l'échelle

Nous avons défini des bornes strictes pour le groupe :

- **Capacité minimale et souhaitée :** 2 instances (pour assurer la haute disponibilité immédiate).
- **Capacité maximale :** 6 instances (pour limiter les coûts en cas d'attaque ou de bug applicatif).
- **Politique de suivi de cible (Target Tracking) :** Nous avons fixé une cible d'**utilisation CPU à 60 %**.
    - *Si le CPU dépasse 60 % :* Auto Scaling ajoute des instances.
    - *Si le CPU redescend :* Auto Scaling supprime les instances superflues.

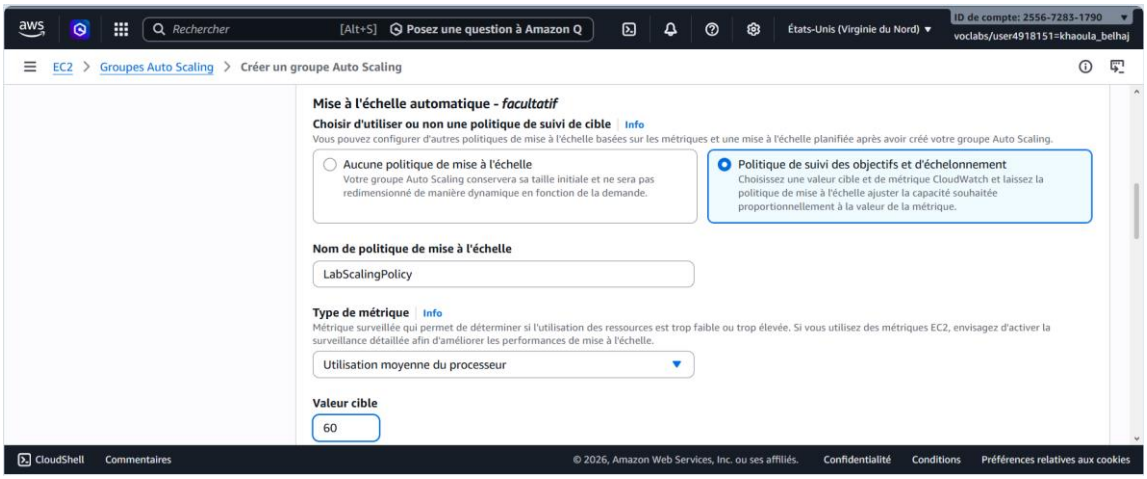


Figure 6 Utilisation CPU à 60 %

4. Intégration et Notification

- **Liaison ELB :** Le groupe a été rattaché au `LabGroup` (Groupe cible). Ainsi, chaque nouvelle instance lancée est automatiquement enregistrée auprès de l'équilibreur de charge.
- **Identifications (Tags) :** L'ajout de la clé `Name` avec la valeur `Lab Instance` permet d'identifier visuellement et instantanément les machines créées par l'automate dans la console EC2.

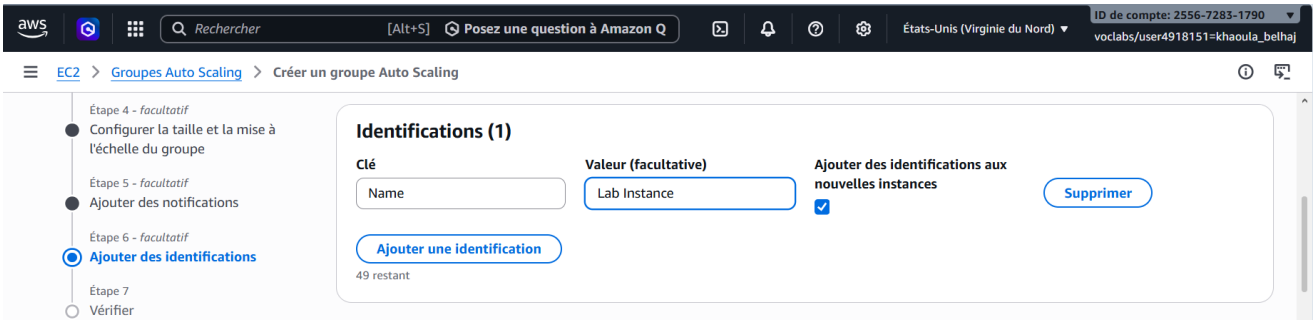


Figure 7 Identification

5. Conclusion de la tâche

La Tâche 3 a transformé un modèle statique en un système vivant. À la fin de cette étape, AWS a automatiquement commencé à provisionner les deux premières instances requises. L'infrastructure est désormais prête à affronter des variations de trafic imprévues sans aucune intervention humaine, illustrant parfaitement le concept de **Cloud Elasticity**.

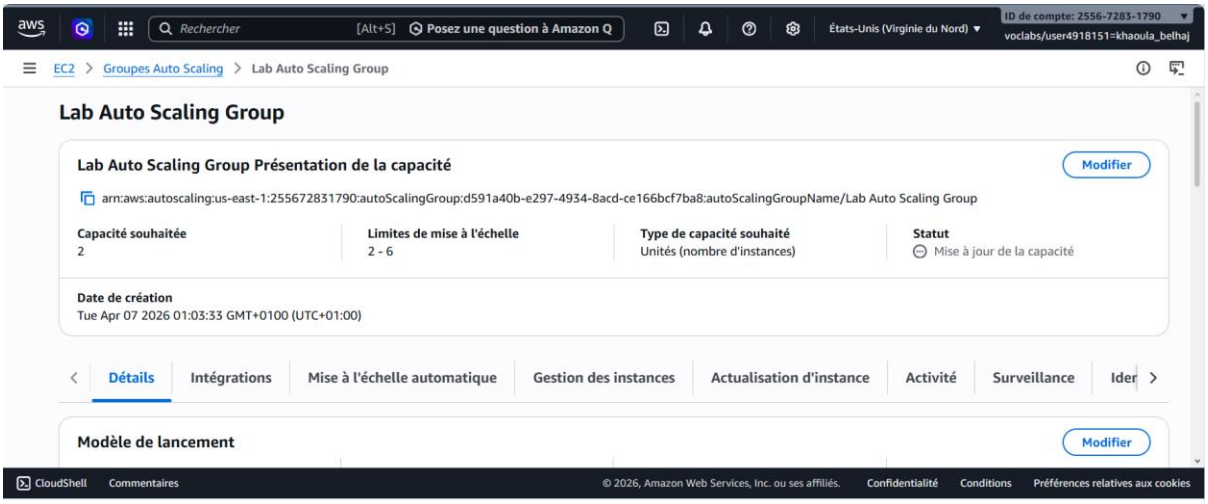


Figure 8 Lab Auto Scaling Group

Tâche 4 : vérification du fonctionnement de l'équilibrage de charge

1. Introduction à la tâche

Après avoir configuré les composants de calcul (Tâche 3) et de réseau (Tâche 2), la Tâche 4 sert de **test d'intégration**. L'objectif est de s'assurer que le flux de trafic est fluide : depuis l'utilisateur sur Internet, en passant par l'équilibreur de charge, jusqu'aux instances privées lancées par Auto Scaling. C'est ici que nous validons que la théorie de la haute disponibilité fonctionne en pratique.

2. Validation des instances et de leur santé

La première vérification s'est faite au niveau du tableau de bord EC2 :

- **Lancement automatique :** Nous avons observé l'apparition de deux nouvelles instances nommées `Lab Instance`. C'est la preuve concrète que le groupe Auto Scaling a respecté sa consigne de "Capacité souhaitée : 2".

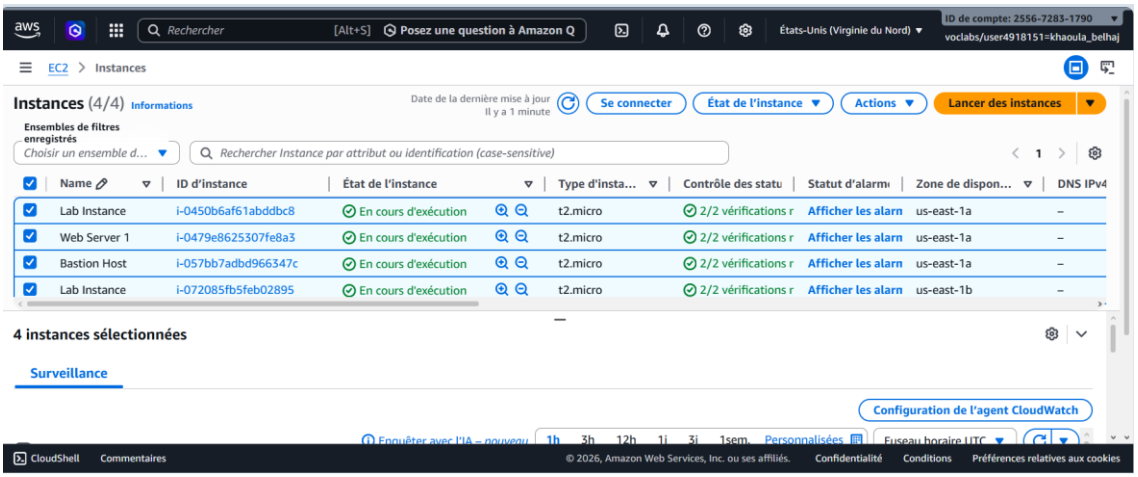


Figure 9 Apparition de deux nouvelles instances

- **Vérification de l'état (Health Checks) :** Dans l'onglet *Cibles* du groupe `LabGroup`, les instances sont passées du statut *Initial* à **Sain (Healthy)**.
    - *Signification :* L'équilibreur de charge a testé avec succès la connexion HTTP sur chaque instance. Il est désormais prêt à leur envoyer des utilisateurs réels.

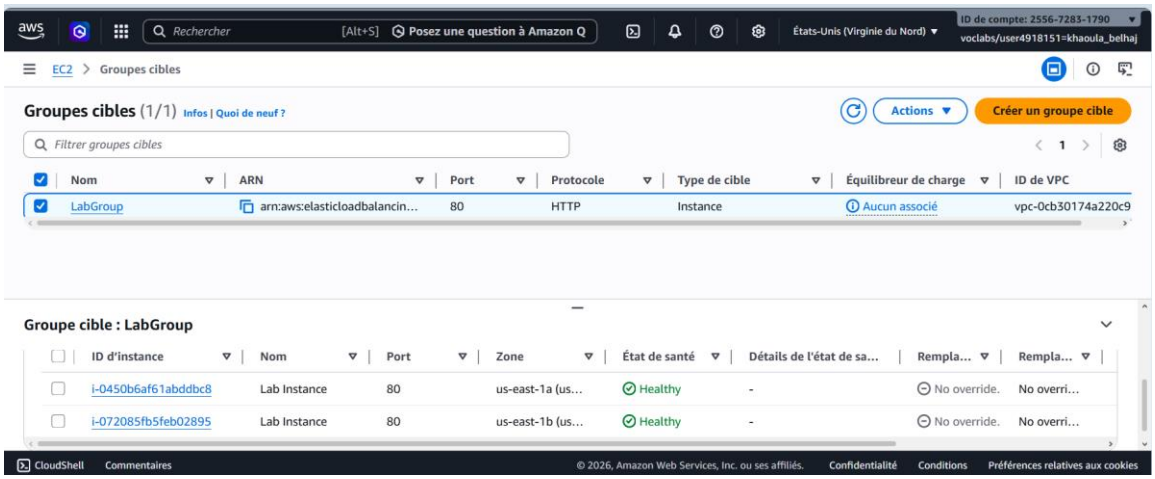


Figure 10 Health Checks

3. Test de connectivité via le nom DNS

L'étape finale a consisté à utiliser le **nom DNS de l'ELB** (*LabELB-683164686.us-east-1.elb.amazonaws.com*) au lieu d'une adresse IP directe.

- **Résultat :** L'affichage de l'application web dans le navigateur confirme que l'équilibreur de charge joue parfaitement son rôle de passerelle.
- **Sécurité confirmée :** Bien que nos instances soient situées dans des sous-réseaux privés (sans IP publiques directes), elles restent accessibles via l'ELB. Cela prouve que notre architecture respecte les meilleures pratiques de sécurité AWS : **isoler les ressources critiques tout en maintenant la disponibilité du service.**

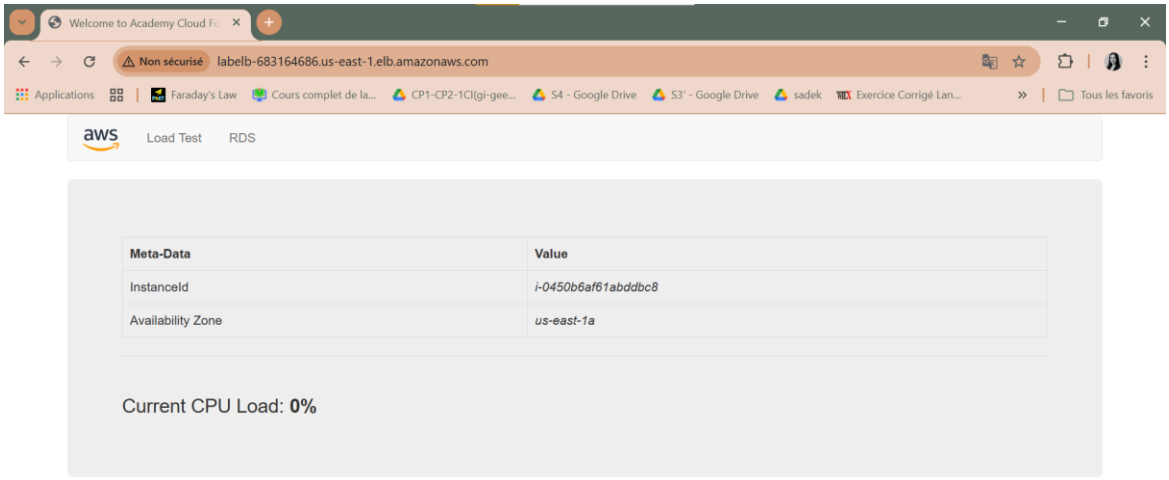


Figure 11 Application web

4. Conclusion de la tâche

La Tâche 4 marque le succès de la mise en réseau. À ce stade, nous avons une infrastructure **stable et redondante**. L'équilibrage de charge est opérationnel, garantissant que le trafic est réparti équitablement entre les deux zones de disponibilité sélectionnées. L'architecture est maintenant prête à subir le test ultime : la montée en charge simulée.

Tâche 5 : test d'Auto Scaling

1. Introduction à la tâche

La Tâche 5 représente la validation finale du dynamisme de l'architecture. L'objectif est de démontrer que le système peut **s'auto-dimensionner** sans intervention humaine. En simulant une activité intense sur le processeur (CPU), nous forçons les alarmes CloudWatch à réagir. Ce test prouve que l'infrastructure est capable de maintenir ses performances même face à un afflux massif et imprévu d'utilisateurs.

2. Déclenchement de l'élasticité : Le Test de Charge

L'opération a été menée en trois phases distinctes :

A. Surveillance via Amazon CloudWatch

Avant le test, nous avons identifié deux alarmes automatiques créées par le groupe Auto Scaling :

- **AlarmHigh :** Se déclenche si l'utilisation moyenne du CPU dépasse **60 %**.
- **AlarmLow :** Se déclenche si l'utilisation du CPU redescend en dessous d'un certain seuil (pour supprimer les instances inutiles).

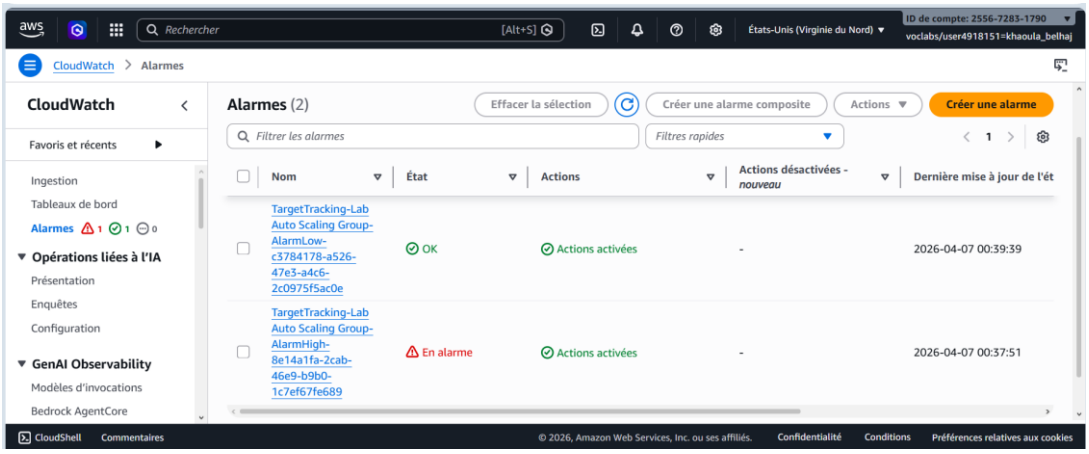


Figure 12 CloudWatch: Alarmes

B. Simulation de la charge (Stress Test)

En utilisant la fonction **"Load Test"** intégrée à l'application web, nous avons généré une activité de calcul intensive sur les instances existantes.

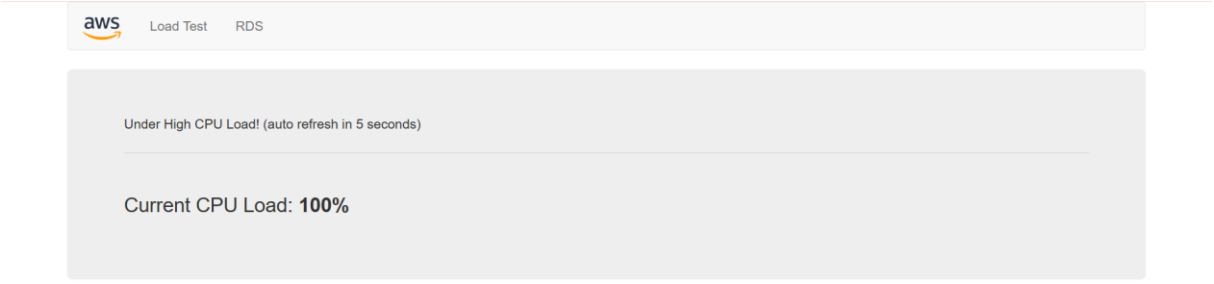


Figure 13 Load Test

- **Observation CloudWatch :** Le graphique de surveillance a montré une courbe ascendante rapide de l'utilisation du processeur.
- **Changement d'état :** L'alarme `AlarmHigh` est passée de l'état `OK` à l'état **`En alarme`** (In Alarm) après avoir maintenu le seuil au-dessus de 60 % pendant la durée définie.

C. Réaction du Groupe Auto Scaling

Dès que l'alarme a été activée, le service Auto Scaling a interprété ce signal pour injecter de la capacité supplémentaire :

- **Provisionnement :** De nouvelles instances `Lab Instance` ont été automatiquement lancées.
- **Résultat :** Le nombre total d'instances est passé de 2 à un nombre supérieur (jusqu'à 6 si nécessaire), répartissant ainsi la charge de calcul pour faire redescendre la moyenne sous le seuil critique.

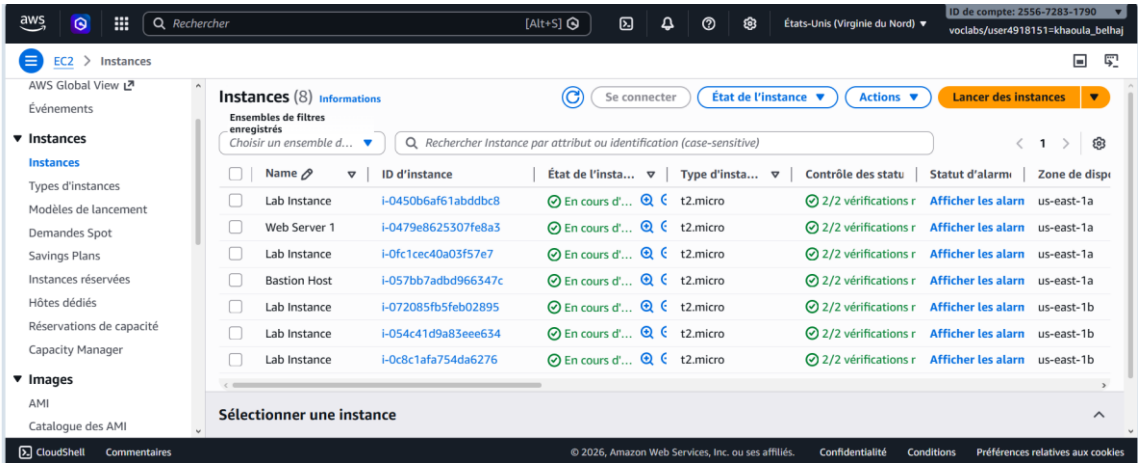


Figure 14 Lab Instance

3. Analyse des résultats

Le test a confirmé que :

1. **La surveillance est précise :** CloudWatch a détecté l'anomalie de charge en temps réel.
2. **L'automatisation est fiable :** Le groupe Auto Scaling a répondu instantanément à l'alarme en lançant des clones parfaits via l'AMI.
3. **La disponibilité est maintenue :** L'application est restée accessible tout au long du processus, prouvant que l'augmentation de la capacité s'est faite de manière transparente pour l'utilisateur.

4. Conclusion de la tâche

La Tâche 5 est le succès technique majeur de cet atelier. Elle illustre la puissance du **suivi de cible (Target Tracking)**. L'infrastructure n'est plus un coût fixe et rigide, mais une ressource fluide qui "respire" avec l'activité de l'entreprise. En automatisant cette réponse, nous éliminons le risque d'indisponibilité lié à une surcharge et garantissons une expérience utilisateur optimale en toute circonstance.

Tâche 6 : résiliation de l'instance Web Server 1

1. Introduction à la tâche

La Tâche 6 est symbolique mais techniquement fondamentale. L'objectif était de supprimer l'instance **Web Server 1**, qui était le serveur d'origine ayant servi à créer l'AMI (Tâche 1). Dans une architecture ancienne, supprimer le serveur principal signifierait la fin du service. Ici, grâce à la mise à l'échelle et à l'équilibrage de charge, nous démontrons que l'infrastructure est devenue **indépendante de ses composants individuels**.

2. Procédure de nettoyage

L'opération a été réalisée avec prudence pour valider la robustesse du système :

- **Identification :** Seule l'instance originale `Web Server 1` a été sélectionnée.
- **Action :** Utilisation de la commande `État de l'instance > Résilier l'instance`.
- **Confirmation :** Une fois l'instance passée à l'état `Terminated`, elle est définitivement supprimée de l'inventaire actif.

3. Analyse du résultat : La "Haute Disponibilité" en action

Pourquoi avons-nous pu supprimer ce serveur sans interrompre l'application ?

1. **L'AMI est le nouveau maître :** Tout le contenu du serveur original est en sécurité dans la `WebServerAMI`.
2. **Auto Scaling prend le relais :** Le groupe Auto Scaling gère désormais ses propres instances (`Lab Instances`) de manière autonome.
3. **L'ELB assure la continuité :** L'équilibreur de charge a simplement cessé d'envoyer du trafic vers `Web Server 1` pour le rediriger exclusivement vers les nouvelles instances saines.

4. Conclusion de la tâche et de l'atelier

La résiliation de `Web Server 1` marque la réussite totale de l'atelier. Nous avons prouvé que l'architecture est désormais :

- **Résiliente :** La perte d'un serveur n'affecte pas l'utilisateur.
- **Élastique :** Elle s'adapte seule à la charge (Tâche 5).
- **Sécurisée :** Les serveurs de production sont isolés dans des sous-réseaux privés.

L'infrastructure est passée d'un état statique et vulnérable à un état dynamique, optimisé pour les performances et les coûts, respectant ainsi les piliers du **AWS Well-Architected Framework**.

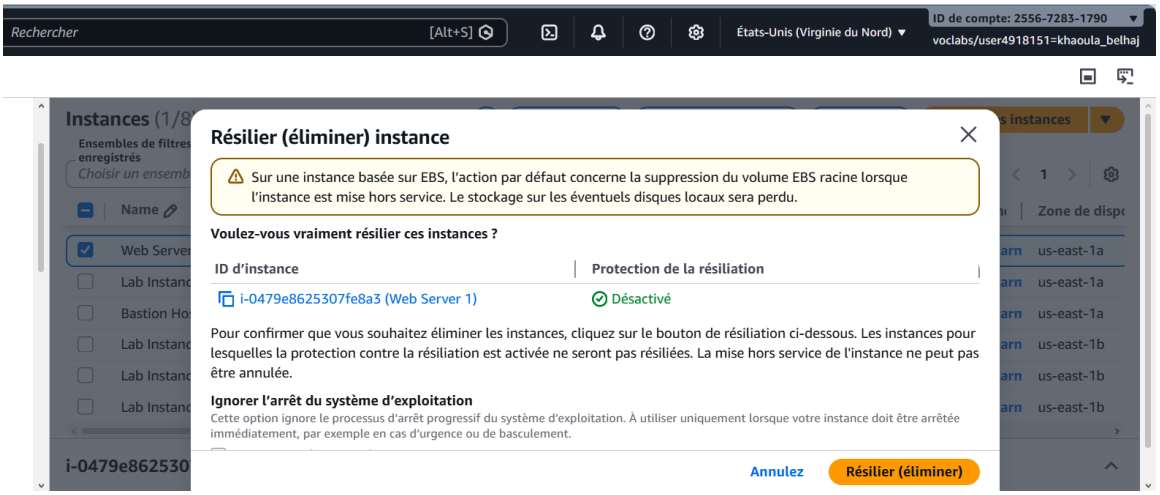


Figure 15 Résilier l'instance

Conclusion

L'achèvement de cet atelier marque une transition fondamentale d'une gestion manuelle des ressources vers une gestion pilotée par les données (data-driven). En isolant les serveurs web dans des sous-réseaux privés et en ne laissant que l'équilibreur de charge exposé publiquement, nous avons non seulement amélioré la **scalabilité**, mais aussi considérablement renforcé la **sécurité** de l'architecture.

La mise en œuvre réussie du groupe Auto Scaling démontre l'agilité du Cloud : le système est désormais capable de s'auto-guérir (en remplaçant une instance défectueuse) et de s'auto-dimensionner sans intervention humaine. Pour une entreprise, cela se traduit par une fiabilité accrue du service client et une optimisation budgétaire rigoureuse, puisque la facture AWS s'ajuste en temps réel à la consommation réelle. En conclusion, la maîtrise de l'ELB et de l'Auto Scaling est la pierre angulaire de toute architecture "Cloud Native" moderne, garantissant que l'infrastructure n'est plus un goulot d'étranglement, mais un moteur de croissance flexible.

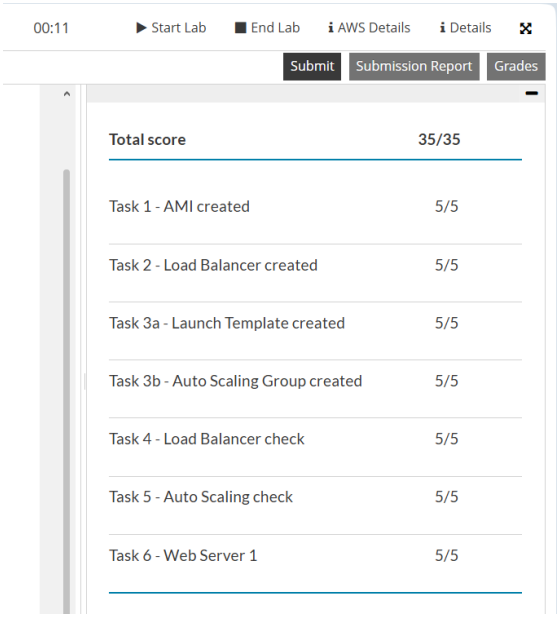


Figure 16 Résultat